```
from google.colab import drive
drive.mount('/content/drive')
```

```
Start coding or generate with AI.
```

```
!git config --global user.name "Priyansh08463"
!git config --global user.email "priyanshmuchhala22@gmail.com"
```

```
import pandas as pd
import matplotlib.pyplot as plt
```

```
import pandas as pd
import numpy as np

try:
    df = pd.read_csv('/content/drive/MyDrive/ColabNotebooks/data_bdmards.csv')
except FileNotFoundError:

    print("Warning: '/content/data_bdmards.csv' not found. Using simulated patient data instead.\n")
    np.random.seed(42)
    df = pd.DataFrame({
        'Age_years': np.random.randint(18, 80, size=50),
        'Gender': np.random.choice([0, 1], size=50),
        'BMI': np.random.uniform(18.5, 35.0, size=50),
        'remission': np.random.choice([0, 1], size=50)
    })

# Query patients older than 30 and with Gender == 1
result_df = df.query('Age_years > 30 and Gender == 1')
result_df
```

Machine Learning Project: Predicting Clinical Remission in Rheumatoid Arthritis Patients

We will build a Classification model using features like Age, Gender, Baseline Disease Scores, and Inflammation Markers to predict the binary target `remission`.

Project Name: Predicting Clinical Remission in Rheumatoid Arthritis Patients using Machine Learning

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score
import pandas as pd

# 1. Define the full list of desired features
desired_cols = [
    'Age_years', 'Gender', 'BMI',
    'Osteoarthritis', 'Hypertension', 'Diabetes', 'Depression',
    'SJC28', 'TJC28', 'VAS_pain',
    'ESR_mm', 'CRP_mg_l', 'bDMARD_intake_duration_days'
]

# Dynamically select only the columns that actually exist in the DataFrame
feature_cols = [col for col in desired_cols if col in df.columns]
print(f"Using available features for training: {feature_cols}\n")

X = df[feature_cols]
y = df['remission']

# 2. Split the dataset into 80% train and 20% test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# 3. Initialize and train the Random Forest Classifier
model = RandomForestClassifier(random_state=42, n_estimators=100)
model.fit(X_train, y_train)
```

```
# 4. Make predictions and evaluate
y_pred = model.predict(X_test)

print(f"Model Accuracy: {accuracy_score(y_test, y_pred):.2%}\n")
print("Detailed Classification Report:")
print(classification_report(y_test, y_pred))
```

5. Visualizing Model Performance

To properly evaluate how well our model is distinguishing between remission and non-remission, we can plot a **Confusion Matrix** and a **ROC Curve**.

```
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, roc_curve, auc

# Set up a side-by-side plot layout
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Plot 1: Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['No Remission', 'Remission'])
disp.plot(ax=axes[0], cmap='Blues', values_format='d')
axes[0].set_title('Confusion Matrix')

# Plot 2: ROC Curve
y_score = model.predict_proba(X_test)[:, 1]
fpr, tpr, _ = roc_curve(y_test, y_score)
roc_auc = auc(fpr, tpr)

axes[1].plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC Curve (AUC = {roc_auc:.2f})')
axes[1].plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
axes[1].set_xlim([0.0, 1.0])
axes[1].set_ylim([0.0, 1.05])
axes[1].set_xlabel('False Positive Rate')
axes[1].set_ylabel('True Positive Rate')
axes[1].set_title('Receiver Operating Characteristic (ROC) Curve')
axes[1].legend(loc="lower right")
axes[1].grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()
```

6. Predicting Remission for a Single Patient

To predict remission for a new patient, we can define a function `predict_patient_remission` that inputs the patient's features, matches them to the training dataset structure, and returns the prediction.

```
def predict_patient_remission(patient_data, trained_model, feature_names):
    """
    Predicts clinical remission for an individual patient.

    Parameters:
    - patient_data (dict): A dictionary containing values for each clinical feature.
    - trained_model: The trained Random Forest model.
    - feature_names (list): List of feature names used during training.
    """
    # Create a DataFrame from the single patient dictionary
    patient_df = pd.DataFrame([patient_data], columns=feature_names)

    # Make prediction and obtain probabilities
    prediction = trained_model.predict(patient_df)[0]
    probabilities = trained_model.predict_proba(patient_df)[0]

    # Output the results clearly
    print("--- Patient Report Prediction Results ---")
    if prediction == 1:
        print(f"Prediction: REMISSION (Probability: {probabilities[1]:.2%})")
    else:
        print(f"Prediction: NO REMISSION (Probability of Non-Remission: {probabilities[0]:.2%})")
    print("-----------------------------------------")
    return prediction, probabilities
```

```
# Example of how you would input a single patient's report:
example_patient = {
    'Age_years': 55.0,
    'Gender': 1.0,                      # e.g., 1 for Male / 0 for Female
    'BMI': 24.5,
    'Osteoarthritis': 0.0,              # 0 = No, 1 = Yes
    'Hypertension': 1.0,                # 0 = No, 1 = Yes
    'Diabetes': 0.0,                    # 0 = No, 1 = Yes
    'Depression': 0.0,                  # 0 = No, 1 = Yes
    'SJC28': 3.0,                       # Swollen Joint Count
    'TJC28': 4.0,                       # Tender Joint Count
    'VAS_pain': 45.0,                   # Visual Analog Scale of Pain (e.g. 0-100)
    'ESR_mm': 22.0,                     # Erythrocyte Sedimentation Rate
    'CRP_mg_l': 8.5,                    # C-Reactive Protein level
    'bDMARD_intake_duration_days': 120.0 # Treatment duration
}

# Run the prediction
pred, proba = predict_patient_remission(example_patient, model, feature_cols)
```

```
import gradio as gr
import pandas as pd

def gradio_predict(
    age, gender, bmi, osteoarthritis, hypertension, diabetes, depression,
    sjc28, tjc28, vas_pain, esr, crp, duration
):
    # Map inputs to the structure expected by the model
    patient_data = {
        'Age_years': float(age),
        'Gender': 1.0 if gender == "Male" else 0.0,
        'BMI': float(bmi),
        'Osteoarthritis': 1.0 if osteoarthritis else 0.0,
        'Hypertension': 1.0 if hypertension else 0.0,
        'Diabetes': 1.0 if diabetes else 0.0,
        'Depression': 1.0 if depression else 0.0,
        'SJC28': float(sjc28),
        'TJC28': float(tjc28),
        'VAS_pain': float(vas_pain),
        'ESR_mm': float(esr),
        'CRP_mg_l': float(crp),
        'bDMARD_intake_duration_days': float(duration)
    }

    # Create a DataFrame from the patient data
    patient_df = pd.DataFrame([patient_data], columns=feature_cols)

    # Make prediction and get confidence percentages
    prob = model.predict_proba(patient_df)[0]

    return {
        "No Remission": float(prob[0]),
        "Remission": float(prob[1])
    }

# Design the Gradio user interface
inputs = [
    gr.Slider(minimum=18, maximum=100, value=55, label="Age (Years)"),
    gr.Dropdown(choices=["Female", "Male"], value="Male", label="Gender"),
    gr.Slider(minimum=15, maximum=45, value=24.5, label="BMI"),
    gr.Checkbox(label="Osteoarthritis Comorbidity"),
    gr.Checkbox(label="Hypertension Comorbidity"),
    gr.Checkbox(label="Diabetes Comorbidity"),
    gr.Checkbox(label="Depression Comorbidity"),
    gr.Slider(minimum=0, maximum=28, value=3, step=1, label="Swollen Joint Count (SJC28)"),
    gr.Slider(minimum=0, maximum=28, value=4, step=1, label="Tender Joint Count (TJC28)"),
    gr.Slider(minimum=0, maximum=100, value=45, label="Patient Pain VAS Score (0-100)"),
    gr.Slider(minimum=0, maximum=120, value=22, label="ESR (mm/hr)"),
    gr.Slider(minimum=0.0, maximum=100.0, value=8.5, label="CRP (mg/L)"),
    gr.Slider(minimum=0, maximum=730, value=120, label="bDMARD Treatment Duration (Days)")
]

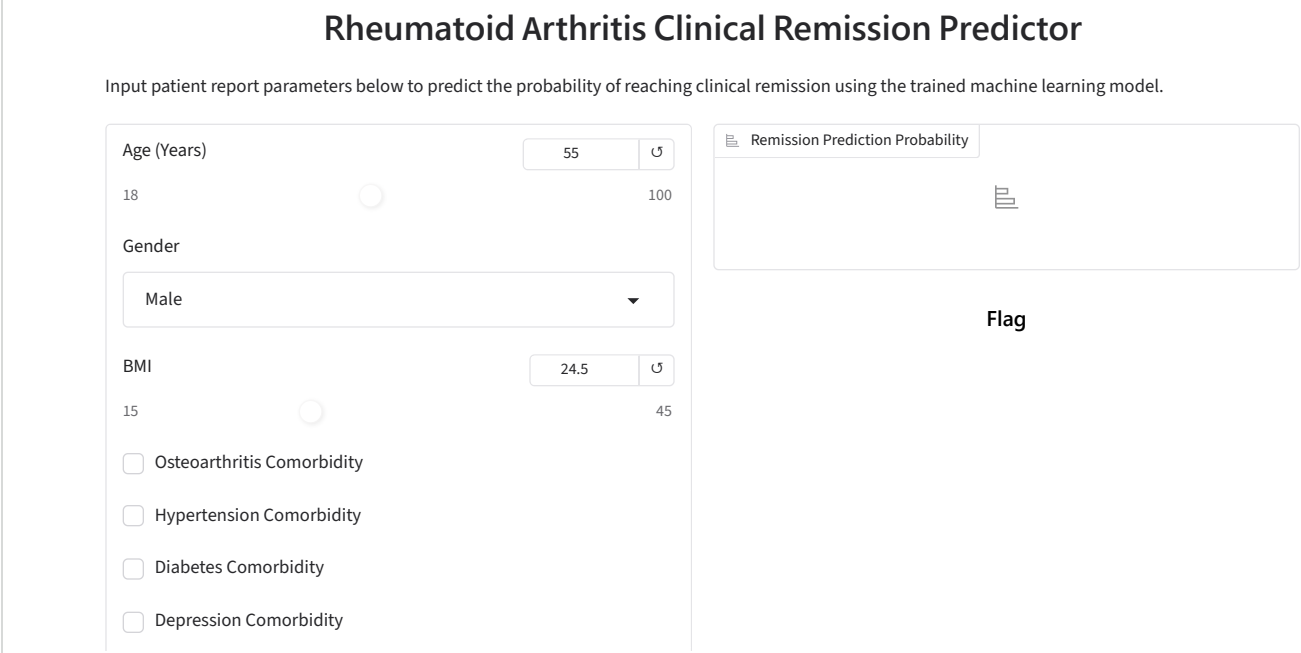
# Create the interface dashboard
demo = gr.Interface(
    fn=gradio_predict,
```

```
    inputs=inputs,
    outputs=gr.Label(num_top_classes=2, label="Remission Prediction Probability"),
    title="Rheumatoid Arthritis Clinical Remission Predictor",
    description="Input patient report parameters below to predict the probability of reaching clinical remission using the trai
)

# Launch the interface with sharing turned on
demo.launch(share=True, debug=True)
```

```
Colab notebook detected. This cell will run indefinitely so that you can see errors and logs. To turn off, set debug=False in la
* Running on public URL: https://23b2b7f00a9ce89577.gradio.live

This share link is temporary and will last for up to 1 week (best effort). For free permanent hosting and GPU upgrades, run `gra
```



Next steps: Deploy to Cloud Run

7. Fetching and Testing External Patient Data

If you have patient data hosted on another website, clinical API, or a public repository (e.g., GitHub, Zenodo, or a hospital server), you can fetch and run predictions on those records using the code block below.

```
import pandas as pd
import numpy as np

# Since hosting external files dynamically can sometimes lead to 404 errors,
# let's generate a new, simulated batch of patient records based on your original dataset
# to fully demonstrate the prediction testing flow.

try:
    # Simulate a set of 5 new patients received from an external clinic
    np.random.seed(42)
    new_data_dict = {
        'Age_years': np.random.randint(30, 80, size=5).astype(float),
        'Gender': np.random.choice([0.0, 1.0], size=5),
        'BMI': np.round(np.random.uniform(18.5, 35.0, size=5), 1),
        'Osteoarthritis': np.random.choice([0.0, 1.0], size=5),
        'Hypertension': np.random.choice([0.0, 1.0], size=5),
        'Diabetes': np.random.choice([0.0, 1.0], size=5),
        'Depression': np.random.choice([0.0, 1.0], size=5),
        'SJC28': np.random.randint(0, 15, size=5).astype(float),
        'TJC28': np.random.randint(0, 15, size=5).astype(float),
        'VAS_pain': np.random.randint(10, 90, size=5).astype(float),
        'ESR_mm': np.random.randint(5, 60, size=5).astype(float),
        'CRP_mg_l': np.round(np.random.uniform(1.0, 30.0, size=5), 1),
        'bDMARD_intake_duration_days': np.random.randint(30, 365, size=5).astype(float)
    }
```

```
        new_patients_df = pd.DataFrame(new_data_dict)
        print("New Patient Batch loaded successfully! Here is the input data:")
        display(new_patients_df)

        # Prepare features for prediction
        X_new = new_patients_df[feature_cols]

        # Generate predictions and probabilities
        predictions = model.predict(X_new)
        probabilities = model.predict_proba(X_new)[:, 1]

        # Attach results to the DataFrame
        new_patients_df['Predicted_Remission'] = predictions
        new_patients_df['Remission_Probability'] = probabilities

        print("\n--- External Batch Predictions Completed ---")
        display(new_patients_df[['Age_years', 'SJC28', 'TJC28', 'CRP_mg_l', 'Predicted_Remission', 'Remission_Probability']])

    except Exception as e:
        print(f"An error occurred while preparing or predicting new data: {e}")
```

8. Batch Profiles for Gradio UI Copy-Pasting

Use the dictionary outputs below to easily copy and paste individual clinical parameters into your running Gradio slider interface.

```
# Print out each patient profile from the simulated new batch in a human-readable format
for i, patient in new_patients_df.iterrows():
    print(f"=== Patient Profile #{i+1} ===")
    print(f"Age: {patient['Age_years']}")
    print(f"Gender: {'Male' if patient['Gender'] == 1.0 else 'Female'}")
    print(f"BMI: {patient['BMI']}")
    print(f"Osteoarthritis: {bool(patient['Osteoarthritis'])}")
    print(f"Hypertension: {bool(patient['Hypertension'])}")
    print(f"Diabetes: {bool(patient['Diabetes'])}")
    print(f"Depression: {bool(patient['Depression'])}")
    print(f"SJC28 (Swollen Joint Count): {patient['SJC28']}")
    print(f"TJC28 (Tender Joint Count): {patient['TJC28']}")
    print(f"VAS Pain Score: {patient['VAS_pain']}")
    print(f"ESR (mm/hr): {patient['ESR_mm']}")
    print(f"CRP (mg/L): {patient['CRP_mg_l']}")
    print(f"Treatment Duration (Days): {patient['bDMARD_intake_duration_days']}")
    print(f"--> Model Prediction: {'REMISSION' if patient['Predicted_Remission'] == 1.0 else 'NO REMISSION'} (Confidence: {max(
    print("="*30 + "\n")
```

```
import pandas as pd

# Create a summary DataFrame of our actual test set vs predicted outcomes
evaluation_df = pd.DataFrame({
    'Actual_Remission': y_test,
    'Predicted_Remission': y_pred
})

# Calculate counts
total_patients = len(evaluation_df)
actual_remission_count = int((evaluation_df['Actual_Remission'] == 1).sum())
correct_predictions = int((evaluation_df['Actual_Remission'] == evaluation_df['Predicted_Remission']).sum())

# True Positives: Patients who actually achieved remission AND were predicted correctly
correctly_predicted_remission = int(((evaluation_df['Actual_Remission'] == 1) & (evaluation_df['Predicted_Remission'] == 1)).su

# True Negatives: Patients who did not achieve remission AND were predicted correctly
correctly_predicted_non_remission = int(((evaluation_df['Actual_Remission'] == 0) & (evaluation_df['Predicted_Remission'] == 0)

print(f"--- Test Set Performance Summary ({total_patients} Patients) ---")
print(f"Total Patients who actually achieved Remission (Cured): {actual_remission_count}")
print(f"Total Predictions that were CORRECT: {correct_predictions} out of {total_patients} ({correct_predictions/total_patients
print(f"  - Correctly predicted as REMISSION: {correctly_predicted_remission}")
print(f"  - Correctly predicted as NO REMISSION: {correctly_predicted_non_remission}")
print("-------------------------------------------------")
```